

NumPy & Pandas Practice Notebook

Summary of Numpy_and_panda_series.ipynb

Overview

This notebook is a hands-on practice exercise covering the fundamentals of two core Python data libraries: NumPy (numerical arrays) and pandas (Series and DataFrames). It progresses from basic array creation to filtering, grouping, and modifying tabular data.

1. NumPy Arrays

1.1 Creating and inspecting a 1D array

A one-dimensional array of daily temperatures (Week 1) is created with `np.array()`, and its core attributes are inspected:

Attribute	Purpose	Result
shape	Dimensions of the array	(7,)
size	Number of elements	7
dtype	Data type of elements	float64
ndim	Number of dimensions	1

1.2 Vectorized operations

The whole array is converted from Celsius to Fahrenheit in a single vectorized expression:

```
temperatures_w1_fahrenheit = (temperatures_w1 * 9/5) + 32
```

1.3 Slicing

Different slice patterns are used to pull out sub-ranges of the array:

- First 3 days: `temperatures_w1[0:3]`
- Last 2 days (weekend): `temperatures_w1[-2:]`
- Middle days: `temperatures_w1[2:5]`

1.4 2D arrays

A second week of temperatures is stacked with the first to form a 2×7 2D array, showing how shape and size scale with dimensions: shape (2, 7), size 14, dtype float64.

2. Pandas Series

A pandas Series named "marks" is created from a list of exam scores with custom string labels (Rank1–Rank5) as the index. Positional access is demonstrated with `.iloc`:

- Single value: `marks.iloc[0]` → 95
- Slice of values: `marks.iloc[0:3]` → first three ranks
- Indexed value: `marks.iloc[2]` → 89

3. Pandas DataFrames

3.1 Building the DataFrame

A "transactions" DataFrame is created from a dictionary with 10 rows and 4 columns: TransactionID, ProductCategory, Region, and Amount.

3.2 Exploring the data

- `.head()` / `.tail()` — preview first/last 5 rows
- `.shape` → (10, 4)
- `.columns` → Index of column names
- `.info` — structural summary of the DataFrame

3.3 Selecting and filtering

- Column subset: `transactions[['ProductCategory', 'Amount']]`
- Last 3 columns: `transactions.iloc[:, -3:]`
- Conditional filter: rows where `Region == 'North'` and `Amount > 200`

3.4 Aggregating

- `value_counts()` on `ProductCategory` → Electronics: 4, Clothing: 3, Furniture: 3
- `unique()` on `Region` → ['North', 'South', 'East', 'West']
- `groupby('Region')['Amount'].mean()` → average Amount per region (East 375.0, North 287.5, South 250.0, West 190.0)

3.5 Modifying the DataFrame

- Update a single cell with `.loc[]` — corrected the Amount for TransactionID 102 from 150 to 165
- Add a new column — `Discount = Amount * 0.10`
- Drop a row by index and drop a column with `.drop()`

4. Key Takeaways

- NumPy arrays support fast, vectorized math and flexible slicing across 1D and 2D shapes.
- pandas Series behave like labeled 1D arrays; `.iloc` gives positional access regardless of the index labels.
- pandas DataFrames support SQL-like operations: selection, boolean filtering, grouping/aggregation, and in-place edits.
- Re-running cells that mutate data in place (drop, assignment) can raise errors if the target no longer exists — a good reason to re-run notebooks from the top when debugging.